

Especificação v2 — Specification Engineering + Open Knowledge Format (OKF) no pipeline do LangNet

Autor: Claude (assistente) · **Data:** 2026-08-12 · **Status:** proposta detalhada para aprovação · **Versão:** 2 (substitui a v1) **Bases:** artigo "Specification Engineering: The New Skill After Prompt Engineering" (KDnuggets, ago/2026) + **Open Knowledge Format (OKF) v0.2** (Google Cloud, jun/2026) **Alvo:** o gerador de apps agênticos do LangNet (backend/agents/langnetagents.py) + etapas do pipeline

0. O que mudou da v1 para a v2

A **v1** propunha 4 inserções de *specification engineering*: **A** (contrato de saída/JSON Schema), **B** (verificação/pós-condições), **C** (spec dos 8 elementos + gap-analysis), **D** (log de suposições).

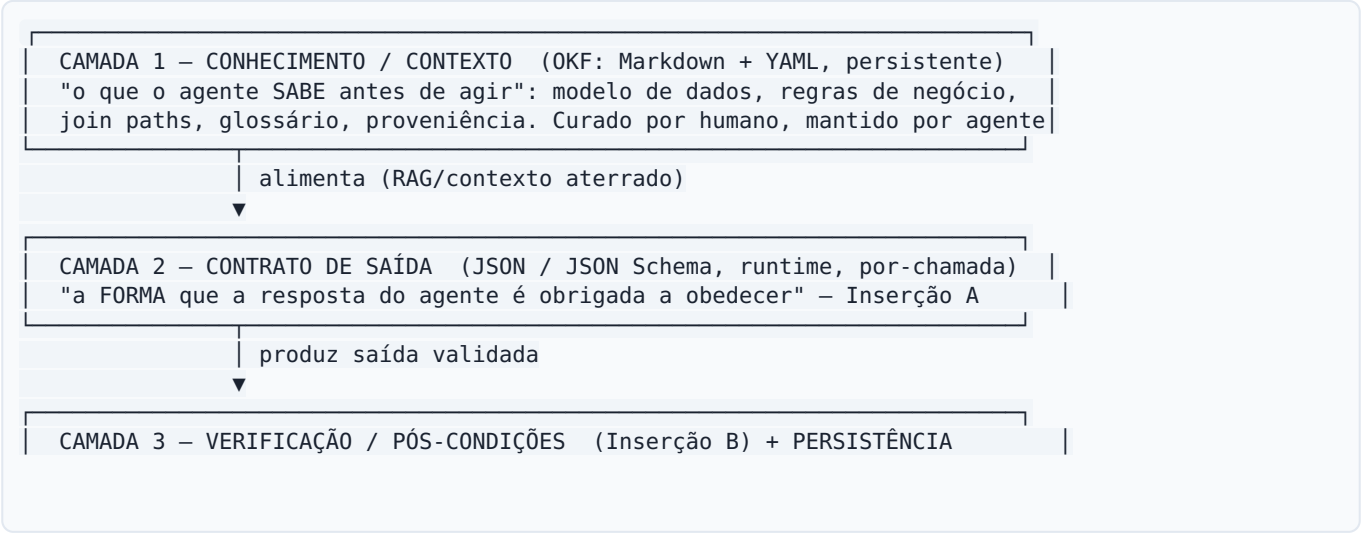
A **v2 mantém A-D** e **acrescenta o OKF** como uma **nova camada** (conhecimento/contexto), com duas inserções: **E** (emitir o domínio como *bundle* OKF que os agentes consomem como contexto aterrado) e **F** (alinhar nossa proveniência/rastreabilidade ao vocabulário OKF). Também explica, numa seção dedicada, **por que OKF não substitui o JSON** — eles vivem em camadas diferentes.

1. As duas referências, em uma frase cada

- **Specification Engineering:** trocar "pedir melhor" por **definir de forma executável e verificável** (objetivo, contexto, inputs, **formato de saída**, restrições, avaliação, edge cases, **verificação**), no workflow especificação → geração → validação → revisão (só do que falhou) → auditoria.
- **Open Knowledge Format (OKF):** representar **conhecimento** como **diretório de Markdown + frontmatter YAML linkados** (um grafo), *portável e legível por humano e agente*, para servir de **contexto** a sistemas RAG/agênticos. Formato, não plataforma (git + texto; sem SDK, sem lock-in). Único campo obrigatório: `type`. A v0.2 adiciona **proveniência** (`sources`), **confiança** (`generated / verified` (convenção de ator), **ciclo de vida** (`status` , `stale_after`) e o tipo **Attested Computation**.

2. A arquitetura em CAMADAS — e por que OKF não substitui o JSON

Esta é a resposta direta à pergunta "o que o OKF substitui? vai substituir o JSON?".



"a saída é ACEITÁVEL? o efeito no banco está correto?"

- **JSON / JSON Schema (Camada 2)** é o **contrato de saída em runtime**: transiente, por-chamada, é a *forma* da resposta do agente. **Continua existindo e é a Inserção A.**
- **OKF (Camada 1)** é **conhecimento persistente**: o *que o agente sabe*. É Markdown+YAML, não JSON, e não tem nada a ver com validar a saída de uma chamada.
- **Eles são complementares, não concorrentes.** O OKF inclusive **se apoia em JSON**: no tipo `Attested Computation`, o `executor.receipt` **declara os campos retornados** (JSON-like), e os `parameters` são tipados. Ou seja, OKF *usa* contratos de campo; não os substitui.

O que o OKF "substitui"/absorve no NOSSO pipeline (nada é apagado — é padronização):

Hoje (ad-hoc)	Com OKF
Injeção de contexto ad-hoc ("DADOS DE ENTRADA..." concatenado no prompt)	Bundle OKF de domínio consumido como contexto aterrado (Inserção E)
Proveniência em colunas próprias (migrations 023-029) + <code>sync-status stale:true/false</code>	Mesmo dado no vocabulário OKF (<code>sources</code> , <code>generated/verified</code> , <code>status</code> , <code>stale_after</code>) — portátil (Inserção F)
Sistema de memória (md + frontmatter + <code>[[links]]</code>)	Já é OKF v0.1 na prática — conformar é trivial

Resposta curta: OKF **não** substitui o JSON. JSON = contrato de saída (runtime). OKF = camada de conhecimento (contexto). O que o OKF substitui é a nossa **injeção de contexto ad-hoc** e o **formato proprietário da proveniência** — de forma aditiva.

3. Mapa: os 8 elementos da spec × as duas referências × nosso pipeline

#	Elemento	Onde EXISTE hoje	Coberto por
1	Objetivo	<code>tasks.yaml</code> → <code>description</code>	— (ok)
2	Contexto	prompt ad-hoc	OKF / Inserção E (bundle de conhecimento)
3	Inputs	<code>description</code> → "Input format" + <code>*_input_func</code>	(ok, validar via A)
4	Formato de saída	<code>expected_output</code> em prosa	Inserção A (JSON Schema)
5	Restrições	esparso	Inserção C
6	Avaliação	Casos de Teste (desconectado)	Inserção B (+ OKF <code>attester</code>)
7	Edge cases	fraco	Inserção C
8	Verificação *	guard parcial	Inserção B (+ OKF <code>Attested Computation</code>)
—	Proveniência/confiança/atualidade	migrations 023-029 + <code>sync-status</code>	OKF / Inserção F

4. Inserção A — Contrato de saída (JSON Schema) por task agêntica + validação/reparo no ws-server

(Camada 2. Peça central de runtime; inalterada da v1, resumida aqui.)

O que é. Cada task agêntica ganha um **JSON Schema de saída**, derivado do `expected_output` e das colunas NOT NULL /tipo da entidade que a task persiste. A saída do agente é **normalizada** → **coagida** → **validada** no ws-server antes de virar `task_completed`; falta de `required` ⇒ **erro explícito** (fail-loud).

Por que. Hoje, em `_template_websocket_server.py` / `_execute_task`:

```
raw = getattr(result, "raw", None) or str(result) # ~L2686
parsed = output_fn(input_data, raw) if callable(output_fn) else {"raw": raw} # ~L2695 ← origem do {raw}
await _send(ws, "task_completed", {"task_name": task_name, "result": parsed}) # ~L2697
```

Sem contrato, manda-se `{raw:...}` cru → daí `parseAgentResult` (frontend) e `_cv` (adapter) como remendos, e campos faltando geram NOT NULL silencioso.

Onde entra. - Gerar: nova função `_derive_output_schema(task_name, task_cfg, entity_model)` em backend/agents/langnetagents.py (parseia `expected_output` + cruza com `_schema_model`; resolve enum↔float pelo tipo do banco). Anexa `output_schema`: a cada task agêntica no `tasks.yaml` gerado (como já fazemos com a nota de coerência). - **Aplicar:** função `_coerce_to_schema(raw, schema)` no template do ws-server, **entre L2695-L2697**: desembrulha `{raw}` /string; coage por tipo (lógica do `_cv`); valida `required`; 1 retry com schema no prompt; senão `error` explícito.

Exemplo (pre_atendimento_cardiologia): `schema {required:[hipoteses,nivel_confianca], nivel_confianca:number(enum→0.4/0.7/0.9), hipoteses:object|array|string→JSON, exames_sugeridos:string}`. Resposta "alta" → 0.9; hipoteses objeto → JSON string; omissão de hipoteses → **rejeita** (não grava meio-registro).

Aposenta: `parseAgentResult` (frontend) e `_cv` disperso (a coerção passa a ser guiada por schema, na fonte). **Esforço:** médio. **Risco:** baixo/médio (modo tolerante por padrão; fail-loud opt-in por task).

5. Inserção B — Checagens de verificação (pós-condições) por task + "revisar só o que falhou"

(Camada 3. Inalterada da v1, resumida.)

O que é. Seção declarativa `verification`: por task (ex.: `not_null:[atendimento_id,medico_id]`, `row_created:encaminhamentos`, `output_has:[hipoteses,nivel_confianca]`), checada **após** a execução (determinística ou agêntica). Falha → `error` com os checks reprovados; refino alimenta **só** o check falho.

Onde entra. Derivar `verification`: das restrições do schema (`_schema_model`); executor `_run_verifications(task, result, input_data)` no ws-server após `det_fn` /agente (L~2631 e L~2697); opção no refino (`app/routers/tasks_yaml.py`, `POST /{sid}/refine`) de "corrija para satisfazer o check X".

Conexão com OKF: uma `verification` é o análogo do **attester** do Attested Computation do OKF (o "código determinístico que verifica a execução"). **Depende da Inserção A** (reusa schema/entidade).

6. Inserção C — Template dos 8 elementos + passo "identificar requisitos faltantes"

(Qualidade upstream. Inalterada da v1, resumida.)

Impor as 8 seções (objetivo/contexto/inputs/output/**constraints/evaluation/edge cases/ verification**) na geração da **Especificação** (`app/routers/specification.py`) e do **Agent-Task Spec** (`app/routers/`

agent_task_spec.py); + passo do agente que **lista requisitos/ambiguidades/assunções** antes de gerar (ROPE: +20% de qualidade quando o requisito é bem articulado). **Esforço:** baixo-médio; aditivo.

7. Inserção D — Log de suposições & limitações (auditoria)

(Passo 6 do workflow. Inalterada da v1, resumida.)

Cada etapa grava `assumptions_and_limitations` (estende a proveniência; exibe na UI). **Nota:** com a Inserção F, esse log passa a viver naturalmente como conteúdo/ `log.md` do bundle OKF. **Esforço:** baixo.

8. Inserção E (nova) — Domínio como bundle OKF consumido pelos agentes (contexto aterrado)

(Camada 1. É a maior novidade da v2 e ataca a alucinação na raiz.)

8.1 O que é

O app gerado passa a incluir uma pasta `knowledge/` no **formato OKF**: um `.md` por conceito do domínio (cada **tabela, métrica/regra de negócio, caso de uso, agente, task**), com frontmatter YAML e **FKs viram wikilinks** entre os arquivos — formando o **grafo de conhecimento** do domínio. Os **agentes do runtime** (ws-server) recebem trechos relevantes desse bundle como **contexto aterrado** (em vez do "DADOS DE ENTRADA..." concatenado hoje).

8.2 Por que é importante

- É a **implementação de referência do próprio Google** aplicada a nós: o *Enrichment Agent* do OKF "varre um dataset, gera um doc OKF por tabela/view e enriquece com schema, join paths e citações" — **exatamente** o que nossa etapa de **Modelo de Dados** já faz (gera schema + descrições).
- **Ataca a classe de bugs de alucinação** que remendamos: o agente inventava sintomas e **consultava histórico_medico (tabela inexistente)**. Com um bundle OKF do schema real (tabelas que existem, com join paths e descrições), o agente tem o **mapa correto do domínio** como contexto — não precisa adivinhar.
- Dá ao app gerado uma **"memória de domínio" portátil** (git, sem lock-in), legível por humano e agente.

8.3 Onde entra (arquivos, funções, artefatos)

- **Produzir o bundle (gerador):** nova função `_emit_okf_bundle(schema_sql, spec_md, tasks_yaml, agents_yaml) -> List[file]` em `backend/agents/langnetagents.py`, chamada onde hoje emitimos `db/schema.sql` e as telas. Gera:
 - `knowledge/index.md` (`okf_version: 0.2` no frontmatter da raiz);
 - `knowledge/tables/<tabela>.md` (frontmatter `type: DB Table, title, description, resource, tags`; corpo com **Schema** (colunas/tipos) e **Joins** com **wikilinks** para as tabelas referenciadas via FK — reaproveita `_schema_model`);
 - `knowledge/use_cases/<uc>.md` (`type: Use Case`, linka as tabelas/tasks que toca);
 - `knowledge/tasks/<task>.md` (`type: Attested Computation` para tasks com adapter determinístico — ver §9);
 - `knowledge/log.md` (histórico).
- **Consumir no runtime (ws-server):** no template `_template_websocket_server.py`, um leitor `_okf_context(task_name, input_data)` que seleciona os `.md` relevantes (pela `entity`/tabelas da task + vizinhos no grafo) e injeta como **contexto** no prompt do agente — **substituindo** o bloco ad-hoc de `_execute_task` (L-2670-2676). Sem RAG pesado: navegação por links (o próprio diferencial do OKF vs similaridade).

- **Opcional (visualização):** o OKF tem um *static HTML visualizer* (1 arquivo) — poderíamos emitir junto para o usuário navegar o grafo de conhecimento do app.

8.4 Exemplo (uma tabela como conceito OKF)

```
---
type: DB Table
title: encaminhamentos
description: Encaminhamento de um atendimento a um médico/especialidade.
resource: mysql://clinia_ops/encaminhamentos
tags: [clinica, atendimento]
generated: { by: "langnet/qwen2.5-coder-32b", at: 2026-08-12T00:00:00Z }
status: stable
---
# Schema
| Coluna | Tipo | Descrição |
|---|---|---|
| atendimento_id | CHAR(36) | FK → [atendimentos](/tables/atendimentos.md) |
| medico_id | CHAR(36) | FK → [medicos](/tables/medicos.md) |
| especialidade_id | CHAR(36) | FK → [especialidades](/tables/especialidades.md) |
# Joins
Liga [atendimentos](/tables/atendimentos.md) a [medicos](/tables/medicos.md).
```

8.5 Esforço / risco

- **Esforço:** médio (emissor determinístico a partir do que já modelamos + leitor de contexto no ws-server).
- **Risco:** baixo (aditivo; é só markdown+git). Ganho direto contra alucinação.

9. Inserção F (nova) — Proveniência / confiança / atualidade no vocabulário OKF

(Camada transversal. Padroniza a rastreabilidade que já temos.)

9.1 O que é

Alinhar nossos registros de **proveniência** e **staleness** ao **frontmatter OKF v0.2**: - **sources** — de quais artefatos/versões um conceito derivou (o que já gravamos: spec vX → protótipo → código), com sinais de credibilidade (author, last_modified). - **generated: {by, at}** e **verified: [{by, at}]** com a **convenção de ator**: - langnet/qwen2.5-coder-32b (agente) — quem **gerou**; - human:pasteur — quem **aprovou** (nosso passo "Aprovar" = verified humano → **trust tier "human-reviewed"**); - process:<pipeline> — geração automática. - **status: draft|stable|deprecated** ← nosso status de sessão (draft/completed/approved). - **stale_after** / staleness ← nosso **sync-status stale:true/false** (quando a spec muda de versão, o artefato derivado fica defasado). O OKF usa **data**; nós usamos **versão** — mapeáveis 1:1.

9.2 Por que é importante

- Torna a **rastreabilidade** (que o usuário faz questão) **padrão e portátil**: "gerado por qwen/..., verificado por human:..., stale_after ..." — sem inventar formato.
- Encaixa nossa detecção de defasagem (stale) num vocabulário reconhecido (consumidores "gate display on stale_after").

9.3 Onde entra

- **Gravar:** ao emitir o bundle OKF (Inserção E), preencher sources / generated / verified / status / stale_after a partir das nossas tabelas de sessão + proveniência (migrations 023-029). Arquivos:

backend/agents/langnetagents.py (emissor) + os routers das etapas (que já têm os dados de versão/aprovação).

- **Ler:** a UI (página de cada etapa) pode passar a exibir o trust tier/staleness no vocabulário OKF.

9.4 Esforço / risco

- **Esforço:** baixo-médio (mapeamento de campos que já temos). **Risco:** baixo (aditivo).

10. Attested Computation — o padrão OKF para o que já convergimos

O tipo **Attested Computation** do OKF descreve uma **computação sancionada**: runtime (ex.: mysql), parameters tipados, computation (o código), executor ({resource, receipt} — o receipt declara os campos retornados), attester ({resource} — código determinístico que verifica a execução). Regra de ouro: **"o agente PODE apenas fornecer valores para os parameters ; NÃO pode escrever/editar a computação"**.

Isso é **exatamente** o que construímos, em ordem inversa: - nossos **adapters <task>_deterministic** = a **computation** sancionada (SQL fixo); - o **dispatch determinístico-primeiro** no ws-server = "roda a computação sancionada, não deixa o agente improvisar"; - o **guard de coerência** + o fix "adapter respeita o input propagado" = garantir que o agente **só fornece parâmetros**; - a **Inserção A (contrato de saída)** = o **receipt** (campos retornados); - a **Inserção B (verificação)** = o **attester**.

Ação (opcional, barata): emitir cada task com adapter determinístico como um conceito type: Attested Computation no bundle OKF (Inserção E), com parameters (do input), receipt (schema da Inserção A) e attester (checks da Inserção B). Assim, **o que inventamos ganha um nome padrão** e fica auto-documentado no grafo de conhecimento.

11. Ordem de implementação recomendada e dependências

A (contrato de saída) → B (verificação; reusa schema de A)
E (bundle OKF de contexto) → F (proveniência OKF; reusa o emissor de E) → (Attested Computation, §10)
C (8 elementos) e D (log) — independentes, baratos, aditivos

1. **A** — maior impacto de runtime (mata a família de bugs de saída); pré-requisito de B.
2. **E** — maior impacto de contexto (mata a família de alucinação); base de F e do §10.
3. **B e F** — fecham verificação e rastreabilidade padronizada.
4. **C e D** — qualidade/auditoria.

MVP sugerido (2 provas na ClinIA): - **A:** gerar output_schema, validar no ws-server, provar que {raw} some e que saída incompleta falha em voz alta. - **E:** emitir knowledge/ OKF do Modelo de Dados e ligar os agentes a ele; provar que o agente para de inventar tabela/sintoma (regressão do bug historico_medico).

12. Impacto esperado — cada dor real × como a v2 previne na fonte

Dor real (já vivida)	Reparo atual	Prevenção v2
{raw:...} no frontend	parseAgentResult	A normaliza no ws-server
enum/ dict em coluna FLOAT/TEXT	_cv	A coage por schema na fonte
campo obrigatório faltando → NOT NULL	best-effort silencioso	A rejeita + B not_null
FK nula persistida	descoberto só no banco	B not_null / row_created

Dor real (já vivida)	Reparo atual	Prevenção v2
agente inventa sintomas / consulta tabela inexistente	injeção ad-hoc + guard	E contexto aterrado (grafo OKF do schema real)
rastreabilidade em formato próprio	migrations + sync-status	F vocabulário OKF portátil

13. Riscos gerais e mitigações

- **Rigidez do contrato (A):** modo tolerante por padrão; `fail-loud` opt-in; `required` só p/ `NOT NULL`.
- **OKF é novo (v0.2, jun/2026), liderado só pelo Google, adoção incipiente, spec evoluindo (v0.1→v0.2 teve breaking):** adotar apenas as **convenções** (frontmatter, actor convention, wikilinks, Attested Computation); **zero acoplamento** a serviços/SDK Google; tratar como camada **opcional/aditiva** (é só `markdown+git` — custo de saída ~nulo).
- **LLM local flaky/lento:** `retry` único guiado por schema/checks ajuda; manter timeouts atuais.
- **Divergência spec↔schema:** A resolve pelo banco; o guard de coerência (já existente) sinaliza o resto.
- **Compatibilidade:** tudo aditivo/opt-in no gerador; regenerar um app aplica as melhorias sem quebrar o protocolo `ws-server↔frontend`.

14. Resumo executivo

Duas referências, **camadas distintas e complementares**: - **JSON/JSON Schema (Inserção A)** — **contrato de saída** em runtime. **Não é substituído pelo OKF**. - **OKF (Inserções E/F)** — **camada de conhecimento/contexto** persistente (`Markdown+YAML`), que *alimenta* os agentes e *padroniza* proveniência/confiança/atualidade.

O **OKF não substitui o JSON**: ele substitui/absorve a nossa **injeção de contexto ad-hoc** e o **formato proprietário da rastreabilidade** — de forma aditiva. Prioridade: **A** (mata bugs de saída) + **E** (mata alucinação por falta de contexto); depois **B/F** (verificação + rastreabilidade padrão), e **C/D** (qualidade/auditoria). Bônus conceitual: nossos adapters determinísticos + A + B já são, na prática, o `Attested Computation` do OKF — a v2 só dá a eles o nome padrão.